

PRACTICAL WEEK – 11

Experiment 1: Multithreaded Counter Using POSIX Threads

Aim

To develop a multithreaded counter application using **POSIX threads** (**pthread_create()** and **pthread_join()**) and demonstrate the **race condition** that occurs when multiple threads concurrently update a shared variable.

Theory

A **thread** is a lightweight unit of execution within a process. Multiple threads can execute concurrently and share the same memory.

In C, **POSIX threads (Pthreads)** are used to create and manage threads.

- **pthread_create()** – creates a new thread.
- **pthread_join()** – waits for a thread to finish execution.
- A **shared variable** is a variable accessed by multiple threads.
- A **race condition** occurs when multiple threads access and modify shared data at the same time, causing unpredictable results.

In this experiment, multiple threads increment a common counter. Since no synchronization mechanism is used, two or more threads may access the counter simultaneously, resulting in an incorrect final value.

Program

```
C counter.c
1  #include <stdio.h>
2  #include <pthread.h>
3
4  #define NUM_THREADS 4
5  #define INCREMENTS 100000
6
7  int counter = 0;
8
9  void *increment_counter(void *arg)
10 {
11     for (int i = 0; i < INCREMENTS; i++)
12     {
13         counter++;
14     }
15
16     return NULL;
17 }
18
19 int main()
20 {
21     pthread_t threads[NUM_THREADS];
22
23     for (int i = 0; i < NUM_THREADS; i++)
24     {
25         pthread_create(&threads[i], NULL, increment_counter, NULL);
26     }
27
28     for (int i = 0; i < NUM_THREADS; i++)
29     {
30         pthread_join(threads[i], NULL);
31     }
32
33     printf("Final Counter Value: %d\n", counter);
34
35     return 0;
36 }
```

OUTPUT:

```
PS C:\Users\panak\Downloads\OSSP PRACTICALS> gcc counter.c -o counter
PS C:\Users\panak\Downloads\OSSP PRACTICALS> counter

Timestamp                CounterSamples
-----
28-09-2026 11:33:40      \\chakrika\network interface(intel[r] wi-fi 6e ax211 160mhz)\bytes total/sec :
123440.999607631

                          \\chakrika\processor(_total)\% processor time :
4.4550311753452

                          \\chakrika\memory\% committed bytes in use :
69.5187321094607

                          \\chakrika\memory\cache faults/sec :
4.97677736147588

                          \\chakrika\physicaldisk(_total)\% disk time :
1.50192801419494

                          \\chakrika\physicaldisk(_total)\current disk queue length :
0
```

Experiment 2: Counter Using Mutex Synchronization

Aim

To modify the multithreaded counter program using a **mutex lock** and compare its output with the race-condition version.

Theory

A **mutex (Mutual Exclusion)** is a synchronization mechanism used in multithreaded programs to control access to shared resources. When multiple threads try to access and modify the same data simultaneously, a **race condition** may occur, leading to incorrect or unpredictable results.

A mutex allows only **one thread at a time** to enter the **critical section**, which is the part of the program where shared data is accessed or modified. Other threads must wait until the mutex is released.

In C, POSIX threads provide the following functions for mutex synchronization:

- `pthread_mutex_init()` – initializes the mutex before it is used.
- `pthread_mutex_lock()` – locks the mutex before entering the critical section.

- `pthread_mutex_unlock()` – unlocks the mutex after completing the critical section.
- `pthread_mutex_destroy()` – destroys the mutex after it is no longer required.

In this experiment, multiple threads increment a common counter variable. The statement `counter++` is placed inside a critical section protected by a mutex. When one thread locks the mutex, other threads cannot modify the counter until the first thread unlocks it.

Thus, the mutex provides **mutual exclusion**, prevents race conditions, maintains **data consistency**, and ensures that the final counter value is correct and predictable.

Program

```
C counter_mutex.c
1  #include <stdio.h>
2  #include <pthread.h>
3
4  #define NUM_THREADS 4
5  #define INCREMENTS 100000
6
7  int counter = 0;
8
9  pthread_mutex_t mutex;
10
11 void *increment_counter(void *arg)
12 {
13     for (int i = 0; i < INCREMENTS; i++)
14     {
15         pthread_mutex_lock(&mutex);
16
17         counter++;
18
19         pthread_mutex_unlock(&mutex);
20     }
21
22     return NULL;
23 }
24
25 int main()
26 {
27     pthread_t threads[NUM_THREADS];
28
29     pthread_mutex_init(&mutex, NULL);
30
31     for (int i = 0; i < NUM_THREADS; i++)
32     {
33         pthread_create(&threads[i], NULL, increment_counter, NULL);
34     }
35
36     for (int i = 0; i < NUM_THREADS; i++)
37     {
38         pthread_join(threads[i], NULL);
39     }
40
41     printf("Final Counter Value: %d\n", counter);
42
43     pthread_mutex_destroy(&mutex);
44
45     return 0;
46 }
```

OUTPUT:

```
● PS C:\Users\panak\Downloads\OSSP PRACTICALS> gcc counter_mutex.c -o counter_mutex
● PS C:\Users\panak\Downloads\OSSP PRACTICALS> .\counter_mutex.exe
Final Counter Value: 400000
○ PS C:\Users\panak\Downloads\OSSP PRACTICALS> 
```

RESULT:

The multithreaded counter was successfully synchronized using a **mutex lock**. The mutex prevented simultaneous access to the shared variable and eliminated the race condition. Therefore, synchronization provides **data consistency and predictable results** in multithreaded programs.